A neural network against an expert rule system

A rule written by hand beats a network trained for 16 epochs, 42.35 against 84.41 bicycles of error

1. What is built

A program that predicts how many bicycles a sharing service will hand out during the next hour. Two answers are compared: a rule written by a person, and a neural network trained on past hours. The rule wins when the network is trained for 16 epochs and loses when it is trained for 512.

The two are then put together in one system: the network answers, except when it predicts a demand above the capacity of the station, and those hours are sent to a person. That is the shape of most systems with a component that learns.

At the end you submit three files: `solution.py`, `execution.txt` and `report.docx`.

2. The data

`bikes.csv` is a CSV file: one hour per line, values separated by commas, column names on the first line. It holds 17,378 consecutive hours of a public bicycle service, from January 2011 to December 2012. These are its first lines:

```
season,hr,holiday,weekday,workingday,weathersit,temp,hum,windspeed,cnt,next_hour
1,0,0,6,0,1,0.24,0.81,0.0,16,40
1,1,0,6,0,1,0.22,0.8,0.0,40,32
```

hr is the hour of the day, from 0 to 23. workingday is 1 when the day is neither weekend nor holiday. weathersit is 1 for clear and grows to 4 for heavy rain. temp, hum and windspeed are the temperature, the humidity and the wind, all three scaled between 0 and 1. cnt is the number of bicycles handed out during that hour, and next_hour is the number handed out during the following one, which is the value to predict.

3. Two things to fix before predicting

The first is the metric. The error is measured in bicycles, as the mean of the distances between the predicted number and the real one:

```
MAE = mean( | predicted - real | )
```

A MAE of 40 means that, on average, the answer misses by 40 bicycles. The mean demand of the hours used for the test is 248.8, so an error of 40 is a miss of about one sixth.

The second is the split. The rows are in chronological order and the split respects that order: the first hours train, the last hours test. Splitting a time series at random puts hours of December in the training set to predict hours of March, which no real service can do, and produces an error that looks excellent and cannot be reproduced on any real day.

hours 1 to 13,902	**hours 13,903 to 17,378**
training, the past	test, the future

Fun fact: Bicing in Barcelona, Velib in Paris and Citi Bike in New York move vans of bicycles between stations during the night, guided by a demand forecast. A forecast that misses by fifty bicycles at the central station is a van sent to the wrong place.

4. Material and setup

The archive `P2_material.zip` on the virtual campus contains `initial_solution.py`, `bikes.csv` and README.md. Unzip it into a folder of your own and open a terminal there.

Check the version of Python and install the two libraries:

```
$ python3 --version
Python 3.11.2
$ pip install --user pandas scikit-learn
```

`initial_solution.py` has four markers, **TODO_1** to **TODO_4**. Each step below fills one of them.

If something fails at this point, it is one of these four:

what the terminal says	what to do
`python3: command not found`	try python instead of python3, and use that name in every command below
`pip: command not found`	use python3 -m pip install --user pandas scikit-learn
`ModuleNotFoundError: No module named pandas`	the install went to another Python; use python3 -m pip install --user pandas scikit-learn
`FileNotFoundError: bikes.csv`	the terminal is in another folder; move to the folder where you unzipped the material

The numbers printed in this handout come from scikit-learn 1.4 with the seed fixed at 42. With another version the last decimal may differ. That is normal and it is not an error: report your own numbers, and check that the order between the answers is the one described here.

5. Work

The steps are done in order and each one keeps the changes of the one before. In every step, replace only the line of the marker and leave the surrounding code untouched. Nothing is undone and the file is never started again from the copy in the archive.

Step 1. Run the program as it is 10 min

From the terminal, in the folder of the practice:

```
$ python3 initial_solution.py
```

It loads the file, splits it in two by the order of the hours, and measures the simplest rule anyone would write, which answers that the next hour will be like this one. It prints:

```
hours in the file: 17378
training hours: 13902
test hours: 3476
mean demand of the test hours: 248.8 bicycles

--- rule, the next hour is like this one ---
mean absolute error: 85.18 bicycles
```

That rule misses by 85 bicycles on average, one third of the mean demand. Copy these numbers into section 3 of `report.docx`.

Step 2. Correct the rule with the hour of the day, TODO_1 25 min

Demand at seven in the morning is not demand at three in the afternoon, and the rule of step 1 ignores that completely. The correction is a factor per hour of the day, measured on the training hours only: for every hour, how much the demand of the following hour grows or shrinks with respect to the current one.

Replace **TODO_1** with these lines:

```
ratio = (pd.DataFrame({"hour": x_train["hr"], "next": y_train, "now": x_train["cnt"]})
         .groupby("hour")
         .apply(lambda group: group["next"].mean() / max(group["now"].mean(), 1),
                include_groups=False))
better_answer = x_test["cnt"].to_numpy() * x_test["hr"].map(ratio).to_numpy()

print("\n--- rule, corrected by the hour of the day ---")
```

One line is missing, the one that prints the error of better_answer. Write it yourself below the others, taking as model the line that prints the error in step 1 and changing the name of the answer.

Run it. The new block prints:

```
--- rule, corrected by the hour of the day ---
mean absolute error: 42.35 bicycles
```

The error falls by half, from 85.18 to 42.35, and not one line of that rule was learnt from the data by a model. Copy this block into section 3 of `report.docx`.

Step 3. Train the network, TODO_2 20 min

A neural network with two hidden layers of 32 units receives the same columns as the rule and learns the answer from the training hours. TRAINING_BUDGET, among the constants at the top of the file, is how many passes over the data it is allowed.

Replace **TODO_2** with these lines:

```
network = MLPRegressor(hidden_layer_sizes=HIDDEN, max_iter=TRAINING_BUDGET,
                       random_state=SEED)
network.fit(x_train_scaled, y_train)
network_answer = network.predict(x_test_scaled)

print("\n--- network trained for %d epochs ---" % TRAINING_BUDGET)
print("mean absolute error: %.2f bicycles" % mean_absolute_error(y_test, network_answer))
```

Run it. Training takes about one second and prints:

```
--- network trained for 16 epochs ---
mean absolute error: 84.41 bicycles
```

The network loses against the rule of step 2, 84.41 against 42.35. Copy this block into section 3 of `report.docx`.

Step 4. Give the network more budget 15 min

`TRAINING_BUDGET` is among the constants at the top of `initial_solution.py`. Change it from 16 to 512 and run again. Training now takes about twenty seconds and prints:

```
--- network trained for 512 epochs ---
mean absolute error: 34.28 bicycles
```

With 32 times the budget the network overtakes the rule, 34.28 against 42.35. Leave the budget at 512 for the rest of the practice. Copy this block into section 3 of `report.docx`.

> **Fun fact:** This is the choice a team makes every week. The rule is understood by everyone, runs in microseconds and is corrected by hand in an afternoon. The network is more accurate and nobody in the room can say why it answered 300 and not 250.

Step 5. Send the busy hours to a person, TODO_3 20 min

A station has a limited number of bicycles. When the predicted demand goes above CAPACITY, which is 400 in this file, the decision is too expensive to be left to a model, and those hours are sent to a person who decides with the whole picture.

Replace **TODO_3** with these lines, which use network_answer, created in step 3 and still available because it belongs to the same execution:

```
automatic = network_answer <= CAPACITY

print("hours answered by the model: %.2f %%" % (100.0 * automatic.mean()))
print("hours sent to a person: %.2f %%" % (100.0 * (~automatic).mean()))
print("mean absolute error of the automatic hours: %.2f bicycles"
      % mean_absolute_error(y_test[automatic], network_answer[automatic]))
```

Run it. It prints:

```
--- model with a person for the busy hours ---
hours answered by the model: 79.69 %
hours sent to a person: 20.31 %
mean absolute error of the automatic hours: 25.88 bicycles
```

The system answers four hours out of five by itself, and on those hours it misses by 25.88 bicycles instead of 34.28. The fifth hour costs the time of a person. Copy this block into section 3 of `report.docx`.

the hours arrive	the network answers	above 400 bicycles
one row per hour	34.28 bicycles of error	a person decides, 20.31 % of the hours

Step 6. Print everything together, TODO_4 15 min

Replace **TODO_4** with these lines, which reuse rule_answer from step 1, better_answer from step 2 and network_answer from step 3:

```
print("answer                          MAE")
print("rule, next hour like this one   %.2f" % mean_absolute_error(y_test, rule_answer))
print("rule corrected by the hour      %.2f" % mean_absolute_error(y_test, better_answer))
print("network                         %.2f" % mean_absolute_error(y_test, network_answer))
```

Run it one last time. The final block prints:

```
--- comparison ---
answer                          MAE
rule, next hour like this one   85.18
rule corrected by the hour      42.35
network                         34.28
```

Copy this block into section 3 of `report.docx`.

Step 7. Save the trace and write the report 30 min

Select everything the terminal printed in the last run, copy it, and paste it into a plain text file named `execution.txt`. Save your finished program as `solution.py`. Open `report_template.docx` and fill in sections 1 to 5: the first three with what you copied in the steps above, section 4 with the error of the first rule, the corrections you made and how the error moved, and section 5 with 100 to 200 words on whether you would ship the rule or the network for a service that has to explain its decisions to the city council. In sections 6, 7 and 8, which are optional, delete the text between square brackets if you leave them empty. Save the file as `report.docx`.

6. Levels

Steps 1 to 7 are the compulsory level. The two below are optional and each one is a single change to a file you already have.

Desirable extension. A second correction for the rule 30 min

The rule of step 2 corrects the demand by the hour of the day, and it ignores that the same hour behaves differently on a working day and on a Sunday. The column workingday says which is which.

Write a third answer that corrects by the two things at once. It is the code of **`TODO_1`** with one change: group by the pair of columns instead of by one, and read the factor with that pair.

```
ratio_two = (pd.DataFrame({"hour": x_train["hr"], "work": x_train["workingday"],
                           "next": y_train, "now": x_train["cnt"]})
             .groupby(["hour", "work"])
             .apply(lambda group: group["next"].mean() / max(group["now"].mean(), 1),
                    include_groups=False))
```

Then build the answer, hour by hour, reading the factor of each pair, and print its error with the same line used in the other rules. Add the result to the comparison of step 6 and report the four numbers in section 6 of `report.docx`:

answer	MAE
rule, next hour like this one	85.18
rule corrected by the hour	42.35
rule corrected by the hour and the working day	yours
network, 512 epochs	34.28

Under the table, state whether the last rule beats the network, and what that would mean for a team that has to explain every prediction to the city council.

Realistic extension. The same system written by a language model 40 min

Install Ollama from ollama.com and download a model, around five gigabytes:

```
$ ollama pull qwen3:8b
$ ollama run qwen3:8b
```

Ask it for the system with a request that states the file, the columns, the split, the model and the metric. For example:

```
Write a Python script with pandas and scikit-learn that reads bikes.csv, uses every column
except next_hour as input and next_hour as target, splits it 80/20 keeping the chronological
order without shuffling, scales the inputs, trains an MLPRegressor with two hidden layers of
32 units for 512 epochs and random_state=42, and prints the mean absolute error on the test
part.
```

Save its answer as `llm_solution.py` and run it without editing it. Report in section 6 of `report.docx` whether it ran at the first attempt, whether its error matches yours, and above all whether it split the data at random or kept the order.

If it does not run, or if it shuffles the rows, do not repair it. Copy the error message or the offending line, ask it once more adding that information to the request, and report what happened both times. A model that fails twice is a result, not a problem, and it is worth as much as one that works.

7. Submission

One ZIP file uploaded through the virtual campus, named with your own surnames and name in lower case and without accents:

```
P2_surname1_surname2_name.zip
```

It contains exactly three files and no folders: `solution.py`, `execution.txt` and `report.docx`. A ZIP with another name, with folders inside or with a different number of files is returned without assessment. If you did any of the two extensions, say so in the corresponding section of the report and do not add files for it.

8. Reference

- Neural networks in scikit-learn: https://scikit-learn.org/stable/modules/neural_networks_supervised.html
- Regression metrics in scikit-learn: https://scikit-learn.org/stable/modules/model_evaluation.html
- Hadi Fanaee-T and Joao Gama. Event labeling combining ensemble detectors and background knowledge. Progress in Artificial Intelligence, 2014.